StudyOS V2 Engineering Blueprint Summary

AI-Native Personal Study Operating System

Purpose

StudyOS V2 should evolve from an AI-powered study application into an AI-native Personal Study Operating System. The objective is not to compete with ChatGPT, Gemini, or Claude on general intelligence, but to become a deeply personalized academic assistant that understands the student's complete learning journey.

Unlike general-purpose LLMs, StudyOS should continuously build long-term academic memory from lectures, uploaded notes, revision history, test performance, mistakes, study habits, and learning preferences. Every AI interaction should become more personalized over time while minimizing API cost through intelligent planning and selective retrieval.

The guiding philosophy is:

> **Think first. Retrieve second. Reason third. Learn continuously.**

Product Vision

The AI should behave like a mentor who has attended every lecture with the student.

It should know:

- Which chapters have been completed.
- Which lecturer taught each concept.
- Which concepts the student understood well.
- Which concepts were confusing.
- Which mistakes are repeatedly made.
- Which topics require revision.
- Which topics are likely to be taught next.
- Which exam (Boards/KCET/JEE) is currently the highest priority.
- How the student prefers explanations.

Instead of answering questions using only an LLM, the system should combine reasoning with long-term academic memory.

Core Architectural Principles

The new architecture should follow these principles:

- Planner-first AI instead of direct RAG.
- Context quality over context quantity.
- Dynamic tool selection.
- Event-driven learning.
- Persistent memory.
- Modular edge services.
- Token-efficient execution.
- Backward compatibility with existing StudyOS features.
- Privacy-first architecture with strict user isolation.
- Every component should be independently replaceable.

The architecture should remain model-agnostic so OpenAI models can later be replaced or supplemented without redesigning the platform.

Overall AI Architecture

Every user request should follow the same lifecycle.

User Request

↓

Intent Understanding

↓

Planner Agent

↓

Tool Selection

↓

Context Engine

↓

Reasoning Model (GPT)

↓

Reflection

↓

Memory Update

↓

Response

The LLM should no longer decide what context to use.

Instead, the Planner Agent becomes the decision-maker while GPT becomes the reasoning engine.

Planner Agent

The Planner Agent is the most important new component.

Its responsibility is to determine:

- What the user is asking.
- Which information is actually required.
- Whether retrieval is necessary.
- Which tools should be called.
- Which memories are relevant.
- How much token budget should be allocated.
- Whether cached knowledge can answer the request.

Example:

Question: "Explain Kirchhoff's Law."

Planner:

- No note retrieval.
- No test retrieval.
- No mistake retrieval.
- Use general reasoning.

Question: "Why did I lose marks in Kirchhoff's Law?"

Planner:

- Retrieve mistakes.
- Retrieve previous tests.
- Retrieve confidence history.
- Retrieve revision history.

Question: "What will Physics sir probably teach tomorrow?"

Planner:

- Lecture timeline.
- Continuity context.
- Syllabus progress.
- Lecturer profile.

No unnecessary retrieval should occur.

Context Engine

The Context Engine should become the heart of StudyOS.

Responsibilities:

- Receive planner decisions.
- Collect only required context.
- Rank retrieved information.
- Remove duplicates.
- Compress long passages.
- Build optimized prompts.
- Respect token budgets.
- Produce consistent prompt structures.

The Context Engine should support adaptive retrieval depth depending on the complexity of the request.

Multi-Layer Memory System

Replace the single learning_memory concept with a hierarchy of memories.

1. Identity Memory

2. Academic profile

3. Subjects
4. Lecturers
5. Exam priorities
6. Preference Memory
7. Preferred explanation style
8. Preferred language
9. Detail level
10. Quiz style
11. Academic Memory
12. Strong topics
13. Weak topics
14. Concept mastery
15. Confidence trends
16. Lecture Memory
17. Lecture timeline
18. Continuity
19. Missed lectures
20. Notes uploaded
21. Mistake Memory
22. Incorrect concepts
23. Repeated errors
24. Misconceptions
25. Confidence changes
26. Revision Memory
27. Revision history
28. Spaced repetition schedule
29. Retention estimates

30. Working Memory
31. Current conversation
32. Temporary context
33. Active goals
34. Reflection Memory
35. Insights extracted after AI interactions
36. New preferences learned
37. Behaviour changes

Each memory type should define:

- Update rules
- Retrieval rules
- Confidence scores
- Expiration policies
- Consolidation strategy

Retrieval Engine

Replace simple vector search with intelligent retrieval.

Support:

- Hierarchical retrieval
- Metadata filtering
- Topic filtering
- Lecture filtering
- Subject filtering
- Time-based filtering
- Semantic retrieval
- Hybrid keyword + vector search
- Ranking
- Context compression

The Planner should decide whether retrieval is needed before any vector search occurs.

Tool Registry

The Planner should orchestrate specialized tools instead of directly querying the database.

Example tools include:

- Search Notes
- Search Tests
- Search Mistakes
- Search Lecture Timeline
- Search Formula Database
- Search Revision Tracker
- Search Performance
- Search Previous Chats
- Search Study Logs
- Search Flashcards
- Search Weak Areas
- Search Academic Memory
- Search User Preferences

Every tool should expose a consistent interface and return structured outputs.

Event-Driven Architecture

Everything the student does should become an event.

Examples:

- Lecture completed
- Topic completed
- Note uploaded
- OCR completed
- Test uploaded
- Question answered
- Wrong answer
- Formula learned
- Revision completed
- Practice session completed
- AI explanation requested
- Flashcards generated

Events should feed analytics, memory updates, recommendations, and planner decisions.

Token Optimization Strategy

Token efficiency should be treated as a first-class design goal.

Implement:

- Planner-based retrieval.
- Semantic caching.
- Prompt compression.
- Adaptive context windows.
- Retrieval only when necessary.
- Chunk ranking.
- Duplicate removal.
- Context summarization.
- Cost-aware planning.
- Incremental retrieval.

The objective is to reduce unnecessary OpenAI usage while improving answer quality.

AI Reflection Layer

Every completed AI interaction should trigger a lightweight reflection step.

Reflection should answer:

- Did the student learn something new?
- Was a misconception detected?
- Should confidence change?
- Should a new memory be created?
- Should revision priority change?
- Has the user's preference changed?

Reflection updates long-term memory automatically.

Knowledge Graph

Introduce an academic knowledge graph linking:

Subjects

↓

Chapters

↓

Topics

↓

Subtopics

↓

Lectures

↓

Notes

↓

Tests

↓

Mistakes

↓

Formulas

↓

Flashcards

↓

Revision Sessions

The graph enables richer retrieval and dependency-aware learning.

Analytics Engine

Analytics should become predictive rather than descriptive.

Provide:

- Mastery scores.
- Retention estimates.
- Weakness trends.
- Revision effectiveness.
- Lecture pace.
- Predicted test performance.
- Suggested daily study plans.
- Learning consistency.
- Concept dependency analysis.

Future AI Capabilities

The architecture should be extensible to support:

- Voice lecture recording.
- Automatic lecture transcription.
- Live classroom assistant.
- OCR improvements.
- Knowledge graph visualization.
- Teacher dashboard.
- Calendar integration.
- Offline support.
- Local LLM integration.
- MCP-compatible external tools.
- Multi-agent collaboration.

Documentation Deliverables

Expand this summary into a complete engineering handbook containing:

1. Product Requirements Document.
2. System Architecture Specification.
3. AI Architecture Specification.
4. Context Engine Design.
5. Planner Agent Design.
6. Memory Architecture.
7. Retrieval Engine.
8. Database Design.
9. Event System.
10. Edge Function Specifications.
11. Prompt Engineering Guide.

12. Token Optimization Strategy.
13. UI/UX Workflows.
14. API Contracts.
15. Migration Plan.
16. Testing & Evaluation.
17. Security & Privacy.
18. Future Roadmap.

Each document should include purpose, architecture diagrams, sequence diagrams, database schemas, APIs, workflows, implementation guidance, acceptance criteria, risks, and future extension points.

The final documentation should be detailed enough for Lovable (or any AI-assisted development platform) to implement StudyOS V2 incrementally while preserving existing functionality and evolving it into a scalable, personalized, AI-native learning platform.